William Jackson

Senior Software Engineer – Python AI/ML

Wellington, Nevada, United States | +1 (786) 949-7561| williamjackson.pro@outlook.com | linkedin.com/in/william-jackson-75b9343b6

PROFESSIONAL SUMMARY

AI Engineer and **Senior Software Engineer** with ~10 years of experience delivering production-grade intelligent systems across **government services**, **health and benefits administration**, and **identity-risk operations**, building **Python 3.10–3.11** platforms that convert unstructured documents, policy logic, and high-volume operational data into reliable, auditable decisions. Strong background in **RAG**, **tool calling**, **FastAPI**, **Celery**, **Redis**, **OpenSearch**, **Kafka**, and workflow orchestration with **Temporal** and **Step Functions**, with a consistent focus on low-latency execution, safe retries, and traceable outcomes. Known for hardening ML and LLM systems through **schema governance**, **evaluation harnesses**, **OpenTelemetry**, **Datadog**, and secure controls for **PII/PHI**, **SOC 2**, and **FedRAMP-readiness** patterns while supporting stable rollout and measurable production impact.

SKILLS

- **Languages** — **Python 3.8–3.11**, **Java 11–17**, **SQL**, **Bash**, **TypeScript 4.x**, Linux fundamentals
- **LLMs, Agents & RAG** — **RAG architectures**, **tool calling**, agent memory, multi-step orchestration, prompt versioning, structured JSON outputs, grounding strategies, retrieval-based response control, safety guardrails, hallucination reduction patterns, golden datasets, rubric scoring
- **Frameworks & Serving** — **FastAPI 0.100–0.110**, **Flask 1.x–2.x**, **Celery 5.x**, **gRPC 1.5x+**, async I/O, SSE, WebSockets, background workers, OpenAPI/Swagger contracts
- **Orchestration & Workflow** — **Temporal 1.x**, **AWS Step Functions**, **EventBridge**, DAG-style processing, long-running workflows, retries, backoff, **idempotency keys**, DLQs, replay-safe execution, human-in-the-loop approvals
- **ML / Data Science** — **scikit-learn 1.x**, **PyTorch 2.x**, **TensorFlow 2.x**, **pandas 1.x–2.x**, **NumPy 1.x**, feature engineering, evaluation pipelines, calibration, drift monitoring, inference optimization, offline/online consistency
- **Retrieval, Search & Vector** — embeddings, chunking strategies, metadata filtering, hybrid retrieval, reranking concepts, **OpenSearch 2.x**, **Elasticsearch 7.x**, analyzers, tokenizers, query tuning, pagination strategies
- **Data & Messaging** — **PostgreSQL 12–15**, **MySQL 8.x**, **SQL Server 2014–2019**, **Redis 5–7**, **MongoDB 4–6**, **DynamoDB**, **Kafka 2.x–3.x**, **RabbitMQ 3.x**, **AWS SQS/SNS**, schema evolution, backpressure controls
- **Cloud & DevOps** — **AWS** (**EKS**, **ECS**, **EC2**, **Lambda**, **API Gateway**, **S3**, **RDS**, **CloudFront**, **IAM**), **Docker 19–24**, **Kubernetes 1.22–1.28**, **Helm 3.x**, **Terraform 1.x**, **CloudFormation**, GitOps patterns, **GitHub Actions**, **GitLab CI**, **Jenkins**, **Azure DevOps**
- **Observability & Reliability** — **OpenTelemetry**, **Datadog**, **Prometheus**, **Grafana**, **CloudWatch**, **ELK/EFK**, structured logging, traces/metrics/log correlation, **SLOs/SLIs**, incident response, postmortems, p95/p99 tuning, capacity planning
- **Security & Compliance** — **OAuth2**, **OpenID Connect**, **JWT**, **Okta**, **Auth0**, **Cognito**, **RBAC/ABAC**, KMS, Secrets Manager, Vault patterns, audit logging, encryption in transit and at rest, **PII/PHI** handling, **SOC 2**, **HIPAA**, **FedRAMP-readiness**
- **Testing & Quality** — **pytest 7.x**, hypothesis, contract tests, integration testing with Docker, load testing patterns, linting gates, migration testing, rollback strategies

WORK HISTORY

Senior Software Engineer
February 2021 to December 2025

Viaro Networks Inc. - Delaware, United States

- Modernized **Python**-based ML and decisioning services by strengthening feature engineering, leakage detection, and label governance, improving model quality by **8–15%** across multiple production use cases without sacrificing existing latency or uptime objectives.
- Improved data trustworthiness by **20–35%** through reproducible **SQL** extraction patterns, Python-based validation gates, and targeted sampling workflows that caught null-heavy and low-agreement records before they polluted training and scoring paths.

- Raised offline-to-online feature consistency by **25–40%** by standardizing point-in-time joins, aligning transformation logic across batch and serving layers, and validating parity between **PostgreSQL**, **DynamoDB**, and streaming event inputs.
- Reduced inference latency by **30–60%** through slimmer feature payloads, strategic **Redis / ElastiCache** caching, and better placement of **FastAPI** inference services across **Lambda** for burst traffic and **EKS/ECS/EC2** for sustained throughput.
- Increased model refresh frequency from monthly to weekly, and in selected workloads from weekly to daily, by automating retraining triggers through **EventBridge** and **Step Functions**, removing manual coordination bottlenecks from production ML workflows.
- Cut model-serving **MTTR** by roughly **35–55%** after introducing **OpenTelemetry** traces, structured logging, and **Datadog**, **Prometheus**, **Grafana**, and **CloudWatch** dashboards with actionable thresholds tied to business and runtime symptoms.
- Reduced production regressions by **40–70%** using CI/CD quality gates, automated backtests, canary rollouts, and rollback-safe deployment controls implemented through **GitHub Actions**, **Terraform**, and **CloudFormation** for Python services.
- Achieved **99%+** success rates on replay and backfill workloads by implementing idempotent consumers for **Kafka** and **SQS/SNS**, solving duplicate write issues and preserving data integrity during retries and historical rebuild operations.
- Enhanced root-cause analysis by indexing prediction artifacts into **Elasticsearch** and **OpenSearch**, enabling slice-based investigation that exposed bias pockets, edge-case degradation, and scoring anomalies that were previously difficult to isolate.
- Strengthened API and platform security with **OAuth2**, **OIDC**, **JWT**, **AWS Cognito**, **RBAC/ABAC**, and detailed audit logging around **FastAPI** endpoints, improving readiness for regulated environments and reducing sensitive-data exposure risk.
- Expanded operational and experiment analytics through **Python/SQL** ELT delivery into **Redshift** and **BigQuery**, giving engineering and product stakeholders better visibility into lift, drift, latency, and infrastructure cost trends.

Software Engineer
October 2018 to January 2021

Avantia, Inc. - Ohio, United States

- Built production ML pipelines using **Python 3.8** and **PostgreSQL 12** snapshots, establishing reproducible datasets and stable training-to-inference parity so model behavior remained explainable across development and runtime environments.
- Resolved a silent feature drift issue caused by inconsistent timezone normalization, standardizing UTC handling and adding timestamp quality assertions that prevented subtle scoring errors from silently compounding in production.
- Implemented a queue-driven scoring service with **Kafka 2.x** consumers and explicit backpressure controls, eliminating memory spikes and preventing OOM failures during periods of unusually heavy ingest volume.
- Migrated fragile legacy batch workloads into containerized worker processes on **Docker 19**, improving environment consistency, reducing deployment friction, and removing frequent “works on my machine” release problems.
- Delivered a new monitoring capability that tracked calibration, segment-level accuracy, and degradation signals over time, helping teams detect weakening model behavior before it became a customer-facing reliability issue.
- Solved a model-version mismatch incident by introducing artifact signing, explicit version headers, and canary rollout policies, which made rollback decisions safer and prevented incompatible model/runtime pairings from reaching broad traffic.
- Added **Redis** caching for hot entity lookups while preserving correctness through TTL controls and change-triggered invalidation hooks, reducing repeated database pressure without introducing stale-read confusion.
- Introduced structured logging and end-to-end request identifiers across services, making incident debugging materially faster even before full tracing coverage had been completed across the platform.
- Modernized CI processes with packaging validation, automated test gates, and dependency checks, reducing risky dependency drift and making builds more reproducible across development, staging, and production environments.

- Built flexible ingestion integrations for multiple external data providers by normalizing schemas at the boundary layer, which kept downstream ML and scoring pipelines stable as upstream partner formats evolved over time.

Software Developer March 2016 to September 2018
ArnAmy, Inc. - Florida, United States

- Built dataset pipelines in **Python 3.6** using **Airflow 1.10**, adding lineage tracking tables and validation checkpoints so analysts and engineers could trust data provenance across recurring training and reporting workflows.
- Fixed a recurring training corruption issue caused by partial files landing on shared network storage, solving it with atomic writes, checksums, and retry-safe staging paths that prevented incomplete artifacts from entering runs.
- Implemented baseline models with **scikit-learn 0.20** and consistent evaluation scripts, creating a more honest measurement framework that replaced ad hoc demo metrics with reproducible comparison standards.
- Added **Elasticsearch 6** indexing for slice-based analysis, allowing engineers to inspect failure clusters and prediction segments quickly without relying on slow, expensive full-table exploration in **Postgres 10**.
- Replaced brittle cron-driven processes with structured **Airflow** DAGs that included retries, dependencies, and alert hooks, which significantly reduced manual overnight recovery work during failed batch cycles.
- Introduced **Redis 4** caching for repeated reference-data lookups, reducing redundant compute and query overhead while keeping correctness intact through deterministic cache keys and explicit invalidation behavior.
- Implemented a new **explainability export** capability that stored top contributing signals for each prediction, making outputs easier for internal stakeholders to review and increasing confidence in model-assisted decisions.
- Shifted fluidly between data pipeline ownership and backend production fixes, helping the team maintain delivery momentum when incident response interrupted roadmap execution and resource priorities changed quickly.

Junior Software Developer October 2014 to February 2016
Centric Tech Inc. - New Jersey, United States

- Built internal **REST** services with consistent validation and error semantics, reducing downstream integration failures and making service behavior more predictable for reporting, operations, and support consumers.
- Fixed a double-processing defect in **RabbitMQ 3.x** workers by tightening ack handling, improving poison-message treatment, and introducing safer retry discipline that prevented duplicate execution in failure scenarios.
- Improved **SQL Server 2014** reporting performance by adding targeted indexes and rewriting heavy join patterns, which reduced timeout risk during finance-close windows and stabilized reporting for business users.
- Added **Mocha 3** and **Chai 3** unit tests around core business rules, increasing regression coverage and catching issues earlier in the release cycle that had previously escaped into weekly deployments.
- Implemented **Redis 3** caching for read-heavy service endpoints, improving response-time stability under load while keeping invalidation rules explicit, reviewable, and simple enough for the whole team to maintain.
- Introduced **Elasticsearch**-powered search endpoints for investigation workflows, replacing slow spreadsheet-driven analysis and enabling faster operational triage during support and audit-related requests.
- Supported incremental upgrades to build and deployment automation, reducing environment drift and making releases more repeatable for a small engineering team still formalizing its delivery process.

EDUCATION

Bachelor's degree in Computer Science 2010 to 2014
Stanford University Department of Computer Science

- Built strong foundations in algorithms, software engineering, and database systems, with academic work that translated well into production-oriented backend development and data-intensive application design.